

This homework has 8 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L<sup>A</sup>T<sub>E</sub>X.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

**Solution:** No collaborators.

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't do last week's homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

**Solution:** I chose the minimum-subarray-product problem from the previous homework as a useful example to reflect on. The key observation is that every nonempty subarray ending at position  $i$  is either the one-element subarray  $[A_i]$ , or is obtained by appending  $A_i$  to a subarray ending at  $i - 1$ . Thus, if  $E_i$  denotes the smallest product of a nonempty subarray ending at  $i$ , then

$$E_i = \min\{A_i, E_{i-1}A_i\}.$$

The answer is  $\min_i E_i$ . A solution can be improved by explicitly separating the local state  $E_i$  from the global answer, stating the base case  $E_1 = A_1$ , and explaining why the two cases above exhaust all subarrays ending at  $i$ . That organization makes both correctness and the  $O(n)$  running time immediate.

2. **Fixed-color-sequence paths.**

Let  $G = (V, E)$  be an undirected graph with  $n$  vertices and  $m$  edges, where  $m \geq n$ . Each edge is colored black or white. Given vertices  $s, t$  and a desired color sequence  $(c_1, \dots, c_k)$ , we must determine whether there is a length- $k$  path from  $s$  to  $t$  with exactly this sequence.

- (10 pts) (a) **Solution:** Define the DP state  $F(i, v)$  to be true exactly when there is a length- $i$  path from  $s$  to  $v$  whose edge colors are  $(c_1, \dots, c_i)$ . The base cases are

$$F(0, s) = \text{true}, \quad F(0, v) = \text{false} \quad (v \neq s).$$

For  $i \geq 1$ ,

$$F(i, v) = \bigvee_{\substack{u \in \text{Adj}(v) \\ \text{color}(u, v) = c_i}} F(i-1, u).$$

The desired answer is  $F(k, t)$ .

This recurrence has the needed optimal-substructure property: the prefix before the last edge must itself be a valid solution to the smaller length- $i-1$  subproblem. To justify the recurrence, consider the last edge of a length- $i$  path ending at  $v$ . It must come from some neighbor  $u$  whose edge has color  $c_i$ , and the preceding edges must form a valid length- $(i-1)$  path from  $s$  to  $u$ . This proves that every valid path is represented by one true term in the disjunction. Conversely, any true term extends a valid length- $(i-1)$  path by the qualifying edge  $(u, v)$ , producing a valid length- $i$  path. The base cases describe the unique length-zero path.

(6 pts)

(b) **Solution:**

```

1: for all  $v \in V$  do
2:    $F[0, v] \leftarrow \text{false}$ 
3:  $F[0, s] \leftarrow \text{true}$ 
4: for  $i \leftarrow 1$  to  $k$  do
5:   for all  $v \in V$  do
6:      $F[i, v] \leftarrow \text{false}$ 
7:     for all  $u \in \text{Adj}(v)$  do
8:       if  $\text{color}(u, v) = c_i$  and  $F[i-1, u]$  then
9:          $F[i, v] \leftarrow \text{true}$ 
10: return  $F[k, t]$ 

```

**Algorithm 1:** FixedColorPath( $G, s, t, c_1, \dots, c_k$ )

This pseudocode evaluates the recurrence by bottom-up table filling. For a fixed  $i$ , the inner adjacency-list scans examine a total of  $2m$  entries, because the graph is undirected and each edge occurs in two adjacency lists. The work for one layer is therefore  $O(m+n) = O(m)$ , since  $m \geq n$ . There are  $k$  layers, so the total running time is  $O(km)$ . The table uses  $O(kn)$  space; keeping only the previous layer would reduce the space to  $O(n)$  without changing the time.

### 3. Back to college

Let  $x_0 = 0$  and  $x_i = \sum_{r=1}^i d_r$  be the total distance from town 0 to town  $i$ .

(7 pts)

(a) **Solution:** Starting immediately after a refueling at town  $p$ , the greedy choice is the farthest town  $q$  that can be reached on the current full pack:

$$q = \max\{r \in \{p+1, \dots, n\} : x_r - x_p \leq R\}.$$

If  $q = n$ , no further refueling is needed. Otherwise refuel at  $q$  and repeat.

```

1:  $x[0] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $x[i] \leftarrow x[i-1] + d_i$ 
4:  $S \leftarrow \emptyset$ ;  $p \leftarrow 0$ 
5: while  $p < n$  do
6:    $q \leftarrow p$ 
7:   while  $q < n$  and  $x[q+1] - x[p] \leq R$  do
8:      $q \leftarrow q + 1$ 
9:   if  $q = n$  then
10:    break
11:    $S \leftarrow S \cup \{q\}$ 
12:    $p \leftarrow q$ 
13: return  $S$ 

```

**Algorithm 2:** GreedyRefuel( $R, D$ )

The prefix sums take  $O(n)$  time. The pointer  $q$  only moves forward, so over all iterations the two pointers together also take  $O(n)$  time. The algorithm uses  $O(n)$  space for the prefix sums, or  $O(1)$  additional space if the distances are scanned with a suitable running total.

(10 pts)

(b) **Solution:** Use the notation from the prompt, and let  $p = i_{\text{diff}} - 1$  be the previous greedy refueling town. Because OPT is valid, it must have a next refueling town after  $p$  unless it can reach town  $n$  directly from  $p$ . The latter case is impossible: the greedy algorithm would then have chosen  $n$  instead of  $i_{\text{diff}}$ . Let  $r$  be the first refueling town of OPT after  $p$ . Since  $i_{\text{diff}}$  is the farthest reachable town from  $p$ ,  $r \leq i_{\text{diff}}$ ; and since  $i_{\text{diff}} \notin \text{OPT}$ , we have  $r < i_{\text{diff}}$ . Let  $j$  be the last town in OPT that occurs after  $p$  and before  $i_{\text{diff}}$ . Thus  $j$  exists and  $j < i_{\text{diff}}$ .

Let  $q$  be the next refueling town of OPT after  $j$ , or let  $q = n$  if there is no such refueling town. Because OPT is valid,

$$x_q - x_j \leq R.$$

The greedy choice from  $p$  satisfies  $x_{i_{\text{diff}}} - x_p \leq R$ . Therefore, replacing  $j$  by  $i_{\text{diff}}$  preserves feasibility on both affected segments:

$$x_{i_{\text{diff}}} - x_p \leq R, \quad x_q - x_{i_{\text{diff}}} \leq x_q - x_j \leq R,$$

where the second inequality uses  $j < i_{\text{diff}} \leq q$ . All other consecutive refueling segments are unchanged. Thus

$$\text{OPT}' = (\text{OPT} \setminus \{j\}) \cup \{i_{\text{diff}}\}$$

is a valid solution.

The exchange removes exactly one town and adds exactly one town, so  $|\text{OPT}'| = |\text{OPT}|$ . Since OPT is optimal, no valid solution can use fewer refueling towns, so OPT' is also optimal. It contains the greedy prefix through  $i_{\text{diff}}$ , as required.

- (4 pts) (c) **Solution:** The exchange argument shows that each greedy choice is safe: whenever an optimal solution first disagrees with the greedy solution, it can be modified into another optimal solution that agrees with one more greedy choice. Repeating this argument makes an optimal solution contain the entire greedy sequence. Since the greedy sequence itself is valid, and an optimal solution cannot have fewer refueling towns than a valid greedy solution containing its whole sequence, the two sequences have the same size. Hence the greedy algorithm is optimal.

- (5 pts) (d) **Solution:** The claim is false. Consider

$$R = 10, \quad D = [6, 4, 6], \quad C = [3, 8, 100].$$

The town positions are  $x_0 = 0, x_1 = 6, x_2 = 10, x_3 = 16$ . The greedy algorithm chooses town 2, because it is the farthest town reachable from town 0 on the initial pack. It therefore pays cost  $c_2 = 8$ .

A better solution refuels at town 1: after traveling 6 units, the new pack covers the remaining distance  $4 + 6 = 10$ , so the car reaches town 3 with total refueling cost  $c_1 = 3$ . Thus the greedy solution is not minimum-cost.

The exchange proof from the previous part fails because exchanging town  $j$  for the later town  $i_{\text{diff}}$  preserves the number of refuelings, but it need not preserve or decrease their monetary cost.

#### 4. Dynamic programming for dynamic shortest paths.

Let  $v_{\text{new}}$  be the added vertex.

- (10 pts) (a) **Solution:** For every  $u \in V$ ,

$$D_{\text{new}}(v_{\text{new}}, u) = \min_{(v_{\text{new}}, y) \in E_{\text{new}}} (w(v_{\text{new}}, y) + D(y, u)), \quad (1)$$

and

$$D_{\text{new}}(u, v_{\text{new}}) = \min_{(y, v_{\text{new}}) \in E_{\text{new}}} (D(u, y) + w(y, v_{\text{new}})). \quad (2)$$

Empty minima are interpreted as  $+\infty$ .

To prove (1), take a shortest path from  $v_{\text{new}}$  to  $u$ . Because there is no negative-weight cycle, a shortest path can be chosen simple, so after its first edge it never returns to  $v_{\text{new}}$ . If that first edge is  $(v_{\text{new}}, y)$ , the remaining path lies entirely in the old graph and has length at least  $D(y, u)$ . This proves that every such path has length at least the right-hand side. Conversely, choosing any outgoing edge and concatenating it with an old shortest path from  $y$  to  $u$  gives a valid path of the displayed length, proving equality. The proof of (2) is the same argument applied to the final edge entering  $v_{\text{new}}$ . If  $d^+$  is the number of outgoing new edges, evaluating (1) for all  $u$  takes  $O(nd^+)$  time. If  $d^-$  is the number of incoming new edges, evaluating (2) takes  $O(nd^-)$  time. Since  $d^+ + d^- = O(n)$  in a simple graph, the total is  $O(n^2)$ .

(8 pts) (b) **Solution:** For every  $u, v \in V$ ,

$$D_{\text{new}}(u, v) = \min \{D(u, v), D_{\text{new}}(u, v_{\text{new}}) + D_{\text{new}}(v_{\text{new}}, v)\}. \quad (3)$$

This is the same shortest-path recurrence idea used in dynamic programming for graphs. A shortest path from  $u$  to  $v$  either avoids  $v_{\text{new}}$ , in which case its length is at least  $D(u, v)$ , or uses it. In the latter case, split the path at its occurrence of  $v_{\text{new}}$ . The first part has length at least  $D_{\text{new}}(u, v_{\text{new}})$  and the second part has length at least  $D_{\text{new}}(v_{\text{new}}, v)$ , so its length is at least the second term of (3). This proves that every path has length at least the right-hand side. Conversely, each term in (3) is realized by a valid old-graph path or by concatenating two valid paths through the new vertex, so the minimum is attainable. If a concatenation repeats a vertex, it is still a valid walk; since there is no negative cycle, any repeated nonnegative cycle can be removed without increasing its length.

The two quantities in (3) have already been computed in the previous part, so each pair  $(u, v)$  takes  $O(1)$  time. There are  $n^2$  pairs, giving  $O(n^2)$  additional time and therefore  $O(n^2)$  total time.

## 5. MST mashup.

(5 pts) (a) **Solution:** The problematic implication is the final one: “Kruskal has one output; therefore that output is the only minimum spanning tree.” Correctness of Kruskal says that its output is an MST, not that every MST must be the output of a particular deterministic algorithm. An algorithm can make a single deterministic choice even when several different optimal solutions exist. Thus uniqueness of Kruskal’s execution does not imply uniqueness of the set of all MSTs.

(8 pts) (b) **Solution:** Suppose for contradiction that two distinct spanning trees  $T_1$  and  $T_2$  are both minimum spanning trees. Choose an edge  $e$  of minimum weight in the symmetric difference  $T_1 \triangle T_2$ , and without loss of generality assume  $e \in T_1 \setminus T_2$ . Remove  $e$  from  $T_1$ . This creates two connected components, say  $A$  and  $B$ . Consider the unique  $T_2$ -path between the endpoints of  $e$ . Its endpoints lie on opposite sides of  $(A, B)$ , so the path contains an edge  $f$  crossing this cut. The edge  $f$  cannot belong to  $T_1 - e$ , since that graph has no edge crossing the cut, and it is not  $e$  because  $e \notin T_2$ . Hence  $f \in T_2 \setminus T_1$ , so  $f$  also belongs to the symmetric difference. All edge weights are distinct, and  $e$  was chosen as the minimum-weight edge in the symmetric difference. Therefore  $w(e) < w(f)$ . Now Adding  $e$  to  $T_2$  creates the unique cycle formed by  $e$  and the  $T_2$ -path just considered;  $f$  lies on this cycle. Therefore  $T_2 - f + e$  is a spanning tree. Its weight is

$$w(T_2) - w(f) + w(e) < w(T_2),$$

contradicting the minimality of  $T_2$ . Thus a graph with distinct edge weights has at most one MST; since a spanning tree exists in the setting of the claim, its MST is unique.

6. Greedy knapsack filling.

- (2 pts) (a) **Solution:** Every feasible 0–1 solution is also a feasible fractional solution by choosing each fraction  $p_i$  to be either 0 or 1. The fractional problem maximizes over a superset of the 0–1 feasible solutions, so its optimal value is at least as large.

- (5 pts) (b) **Solution:** The proposed algorithm is not correct. Consider capacity  $C = 10$  and three items:

$$(w_1, v_1) = (100, 100), \quad (w_2, v_2) = (1, 1), \quad (w_3, v_3) = (10, 90).$$

VALUEGREEDY first chooses item 1, since it has value 100. It takes a  $1/10$  fraction and obtains value 10.

WEIGHTGREEDY first takes all of item 2, using one unit of capacity. It then takes a  $9/10$  fraction of item 3, obtaining total value  $1 + 81 = 82$ . Thus the better of the two greedy outputs has value 82.

However, taking all of item 3 uses exactly the capacity and gives value 90, which is optimal: item 3 alone has value 90, and no feasible solution can use more than the maximum value-per-weight item 3 on all ten units of capacity. Therefore the proposed algorithm can return a suboptimal answer.

- (10 pts) (c) **Solution:** Let  $r_i = v_i/w_i$  and assume  $r_1 \geq r_2 \geq \dots \geq r_n$ . Let  $p_i$  be the fractions selected by RELATIVELYGREEDY, and let  $q_i$  be the fractions of an arbitrary optimal solution OPT. Choose the smallest index  $j$  for which  $q_j \neq p_j$ . The fractions for indices below  $j$  already agree, so let

$$C' = C - \sum_{i < j} p_i w_i$$

be the capacity remaining before item  $j$ .

First suppose  $p_j < 1$ . Then the greedy algorithm fills the remaining capacity with item  $j$ , so  $p_j w_j = C'$  (the case  $p_j = 0$  cannot be a first mismatch with a feasible solution unless there is no remaining capacity). Feasibility of OPT implies

$$q_j w_j + \sum_{i > j} q_i w_i \leq C' = p_j w_j,$$

and hence  $q_j \leq p_j$ . Let  $\delta = p_j - q_j > 0$ . There is at least  $\delta w_j$  of capacity available either as unused capacity or as weight currently assigned to later items. Add a  $\delta$ -fraction of item  $j$ , and remove a total weight  $\delta w_j$  from later selected items, using unused capacity first and then reducing later fractions as needed. This yields a feasible solution OPT' with the required fraction of item  $j$ .

Now suppose  $p_j = 1$ . Then  $C' \geq w_j$ . Since  $q_j \neq 1$  and  $0 \leq q_j \leq 1$ , let  $\delta = 1 - q_j > 0$ . The quantity of unused capacity plus the weight assigned to later items is

$$C' - q_j w_j \geq w_j - q_j w_j = \delta w_j,$$

so the same exchange is possible: add a  $\delta$ -fraction of item  $j$  and remove total weight  $\delta w_j$  from unused capacity and later items.

In either case, every removed fraction belongs to an item  $i > j$ , whose relative value satisfies  $r_i \leq r_j$ . The value added is  $\delta w_j r_j$ . The value removed from later items is at most  $\delta w_j r_j$ , and unused capacity removes no value. Consequently

$$\text{value}(\text{OPT}') \geq \text{value}(\text{OPT}).$$

Since OPT was optimal, OPT' is also optimal. It agrees with the greedy fractions through index  $j$ . Repeating this exchange for the next disagreement eventually transforms an optimal solution into the greedy solution. Therefore RELATIVELYGREEDY is correct.

## 7. Optional extra credit: Greedily independent.

(1 EC pt)

(a) **Solution:** Use vertices  $\{1, 2, 3, 4, 5, 6\}$  and edges

$$\{(1, 2), (1, 3), (1, 4), (2, 5), (2, 6), (3, 5), (3, 6), (4, 5), (4, 6), (5, 6)\}.$$

Vertex 1 has degree 3, vertices 2, 3, 4 have degree 3, and vertices 5, 6 have degree 5. Therefore the tie-breaking rule selects vertex 1 first. It removes vertices 2, 3, 4, after which the algorithm selects one of 5, 6, returning an independent set of size 2. But  $\{2, 3, 4\}$  is an independent set of size 3, so the output is not maximum.

(2 EC pts)

(b) **Solution:** Consider the graph remaining immediately before the algorithm selects  $v_{\text{diff}}$ . It is a forest, because deleting vertices and edges from a tree leaves a forest. Every nonempty forest has a vertex of degree at most 1, so the minimum-degree choice satisfies  $\deg(v_{\text{diff}}) \leq 1$  in the remaining forest.

All previously chosen greedy vertices belong to OPT by the setup, and the remaining vertex  $v_{\text{diff}}$  is not adjacent to any of them; otherwise it would already have been deleted. If  $\deg(v_{\text{diff}}) = 0$ , then  $v_{\text{diff}}$  has no neighbor in OPT, so we could add it to OPT, contradicting maximality of OPT. Thus it has exactly one remaining neighbor, call it  $u$ , and  $u \in \text{OPT}$ .

Define

$$\text{OPT}' = (\text{OPT} \setminus \{u\}) \cup \{v_{\text{diff}}\}.$$

The only possible neighbor of  $v_{\text{diff}}$  in the remaining forest is  $u$ . Any vertex deleted earlier is either a previously chosen greedy vertex (which is in OPT and is not adjacent to  $v_{\text{diff}}$ ) or a neighbor of one of those vertices (and hence cannot belong to the independent set OPT). Therefore adding  $v_{\text{diff}}$  after removing  $u$  creates no adjacency inside OPT'. The set is independent and has the same cardinality as OPT, so it is also a maximum independent set. It contains the greedy prefix through  $v_{\text{diff}}$ .

Repeating the exchange for each first disagreement produces a maximum independent set containing the entire greedy output. Since the greedy output itself is independent, it must have the same size as a maximum independent set and is therefore maximum.